TRIGGERS AND SHIFT REGISTERS

Purpose of the work

Learn about the simplest memory elements - triggers (their types, properties and implementation using logic elements) and shift registers.

Laboratory work task

1. Design a static control trigger with its control schemes according to the characteristic equation given in the table of individual tasks, using logic elements. Investigate its performance.
2. Design synchronous Master-Slave and dynamic triggers using the same control schemes. Investigate their performance.
3. Design a universal shift register of the specified length. Investigate its performance.
4. Upload the shift register diagram to PLIS and demonstrate how the shift register works.
5. Prepare a lab report. The report should present the design process, the decisions made during the design process and the results of the simulation of these schemes with their comments.

Example

1. Design a static trigger with control schemes described by the expression

$$(x_1 + x_1 (\overline{x_2} - x_3) + x_4)\overline{Q}_t + x_1 \overline{(}(x_3 \oplus x_4) - x_2);$$

We write this expression in the form of a trigger characteristic equation:

$$Q_{t+1} = (x_1 + x_1 ((\overline{x_2} - x_3) + x_4))\overline{Q}_t + x_1 ((x_3 \oplus x_4) - x_2) Q_t$$

and compare it with the characteristic equations of the triggers:

SR trigger: $Q_{t+1} = \overline{C}Q_t + C\overline{R}Q_t + CS$;

JK trigger: $Q_{t+1} = \overline{C}Q_t + C(\overline{K}Q_t + J\overline{Q}_t)$;

D trigger: $Q_{t+1} = \overline{C}Q_t + CD_t$.

We can see that the expression given in the problem corresponds to the characteristic equation of a synchronous JK trigger, rearranged as follows: $\boldsymbol{Q_{t+1}} = (\boldsymbol{C} + \boldsymbol{CK})\boldsymbol{Q_t} + \boldsymbol{J}\,\boldsymbol{Q_t}$.

So,

$$\boldsymbol{C} = \overline{\boldsymbol{x_1}}\ ;$$

$$\overline{\boldsymbol{K}} = (\boldsymbol{x_2} - \boldsymbol{x_3}) + \overline{\boldsymbol{x_4}}\ ;$$

$$\boldsymbol{J} = (\boldsymbol{x_3} \oplus \boldsymbol{x_4}) - \boldsymbol{x_2}\ .$$

The schematic of a static JK trigger with control circuits and input ***Rst***, which sets the trigger to its initial state ('0'), is shown in Figure 1. Note that we have K in the expressions, so element I28 in the diagram is OR-NE (NOR).

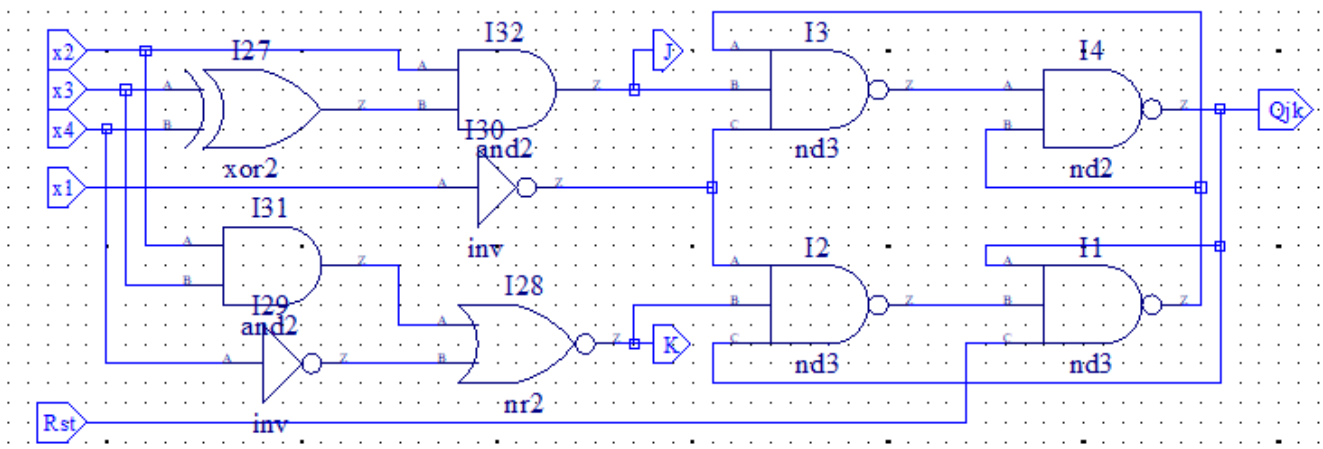


Figure 1. Schematic diagram of the UK trigger with control circuitry

The circuit is complemented by two outputs (J and K) to monitor the values of signals ***J*** and ***K*** (and to facilitate error detection).

The performance of the static JK trigger is described in the following table:

Table 1. SR trigger operating table

J	***K***	$\boldsymbol{Q_{t+1}}$	***Operation***
0	0	$\boldsymbol{Q_t}$	Storage of information
0	1	0	Zero recording
1	0	1	Recording a unit
1	1	x	Prohibited combination [1]

[1] The two-stage and dynamic triggers then change state to the opposite ("translate")

Given the expressions for ***K*** and ***J,*** we construct a correctness table showing the dependence of the ***J*** and ***K*** signals on the signals x_2 , x_3 , x_4 (the forbidden combination of ***J*** and ***K is highlighted*** in red).

Table 2. Dependence of J and K on signals x_2 , x_3 , x_4

x_2 x_3 x_4	*K*	*J*
0 0 0	0	0
0 0 1	1	0
0 1 0	0	0
0 1 1	1	0
1 0 0	0	0
1 0 1	1	1
1 1 0	0	1
1 1 1	0	0

The test must verify the correctness of both the trigger and the control circuitry. To check the trigger's operation, we need to perform all the operations it performs - storing information, writing zeroes and ones. To test the operation of the control circuitry, we need to feed it all allowed combinations of the variables x_2 , x_3 and x_4 .

Taking the data from Tables 1 and 2 into account, we build a sequence of test suites to verify the scheme (see Table 3).

Table 3. Sequence of test suites to verify the scheme

x_2 x_3 x_4	*K*	*J*	Q_{t+1}	*Operation*
1 1 0	0	1	1	Recording a unit
0 0 0	0	0	1	Storage of information
0 0 1	1	0	0	Zero recording
0 1 0	0	0	0	Storage of information
1 1 0	0	0	1	Recording a unit
1 1 1	0	0	1	Storage of information
0 1 1	1	0	0	Zero recording
1 0 0	0	0	1	Storage of information

Let the synchronisation period be 50 ps. Table 8 shows the data for the generation

of signals x_2 , x_3 and x_4 . It shows the times at which the level of each signal changes (from 0 to 1 and from 1 to 0). Since the change has to be done at passive synchrosignal level, less than half a period of 20 ps has been added to the period (50 ps).

Table 4. Data for the generation of signals x2 x3 x4

x_2 x_3 x_4	***Signal change start time***			***Operation***
	x_2	x_3	x_4	
1 1 0	0	0	0	Recording a unit
0 0 0	70	70		Storage of information
0 0 1			120	Zero recording
0 1 0		170	170	Storage of information
1 1 0	220			Recording a unit
1 1 1			270	Storage of information
0 1 1	320			Zero recording
1 0 0	370	370	370	Storage of information

The scheme starts at a passive synchrosignal level; since the synchronisation is performed by an inverse signal ($\overline{x_1}$), the shaping of this signal must be done by a directive (the signal starts at high level):

force -freeze sim:/Lab2_jk/x1 1 0, 0 {25 ps} -r 50

From the data in this table, we form the following modelling directives:

force -freeze sim:/Lab2_jk/x2 1 0, 0 70, 1 220, 0 320, 1 370
force -freeze sim:/Lab2_jk/x3 1 0, 0 70, 1 170, 0 370
force -freeze sim:/Lab2_jk/x4 0 0, 1 120, 0 170, 1 270, 0 370

The Rst signal is generated by the following modelling directive:

force -freeze sim:/Lab2_jk/rst 0 0, 1 {10 ps}

Simulations of the operation of the scheme produced the following timing diagrams:

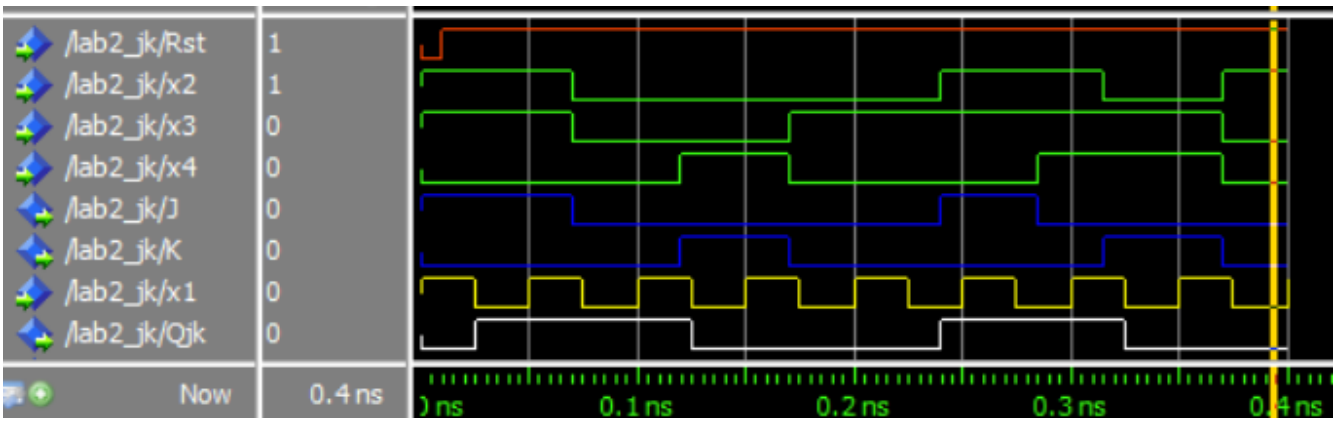


Figure 2: Timing diagram for a scheme with a static JK trigger

2. Now let's design a Master-Slave trigger using the same control schemes (section 2.1).

In the same graphical editor window, we draw a schematic of a Master-Slave (MS) JK trigger with control circuits and an input that sets the trigger to the initial state (0), as shown in Figure 3.

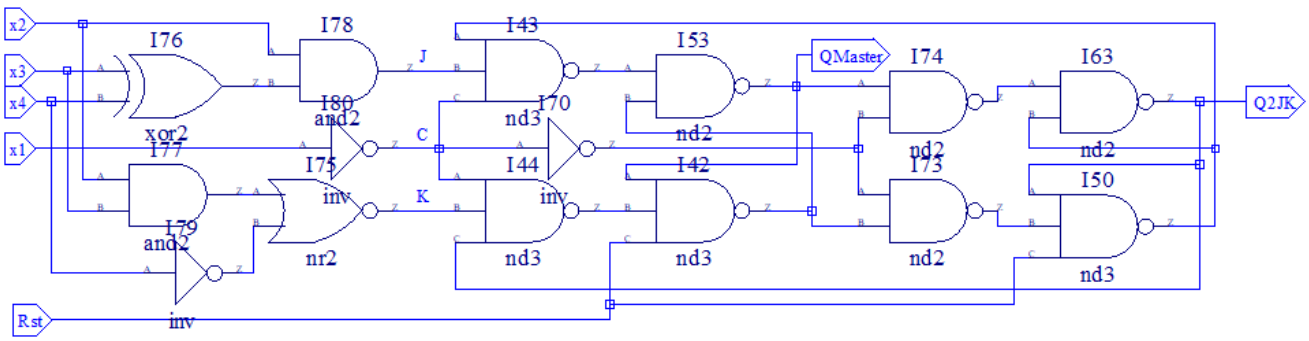


Figure 3: Schematic of a Master-Slave **UK trigger**

To verify the scheme, using a test set constructed by adding at the end of the test set the values of the signals x_2 , x_3 , x_4 forming **J** = **K** = 1, we obtain the following time diagrams:

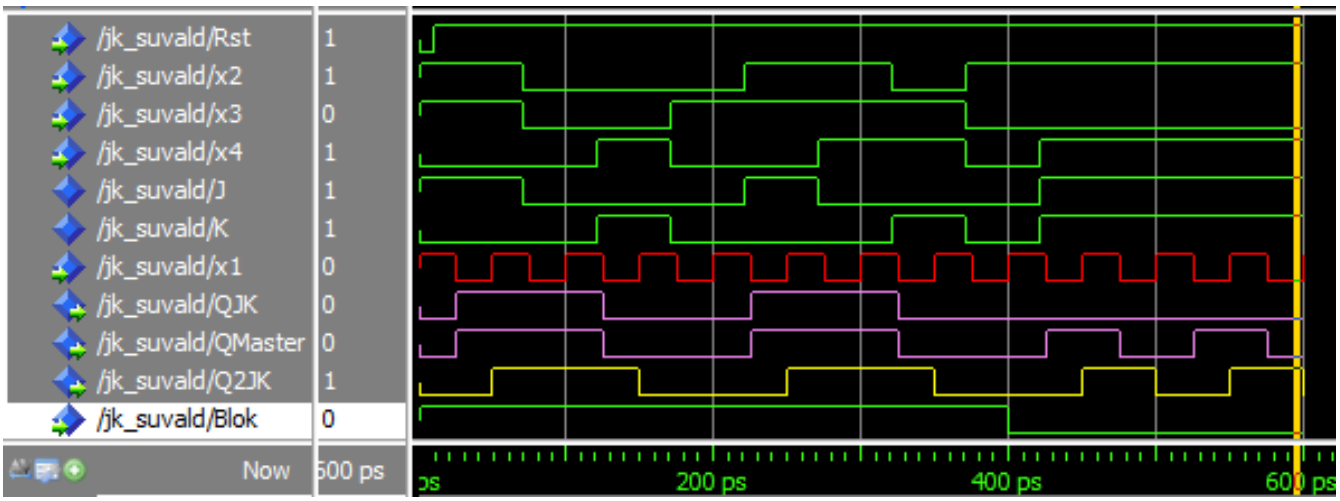


Figure 4: Timing diagram for static and Master-Slave **UK triggers**

We can see that up to 400 ps, the work of the first-stage UK trigger (QMaster) overlaps with that of the static trigger (QJK). The first stage JK trigger switches when ***x1*** goes low and the second stage trigger (Q2JK) replicates the QMaster transitions at the moment when ***x1*** goes high. From 400 ps onwards, the static trigger was stopped by blocking its synchronisation (signal Blok) and the Master-Slave trigger continued to operate. At 420 ps, when **x4** switched to 1, the values of J and K became 1, so that with each synchronisation pulse (**x1**) the Master-Slave JK trigger started to "tick", changing state to the opposite.

Using the same control schemes, let's change the trigger to a dynamic trigger (Figure 5); its timing diagram (along with timing diagrams for other types of UK triggers) is shown in Figure 6.

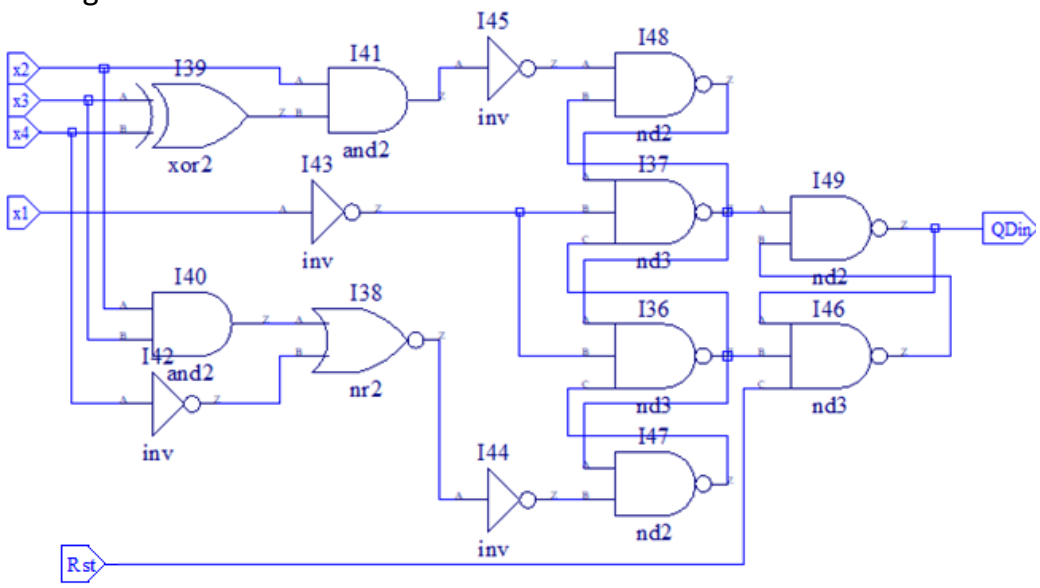


Figure 5: Schematic with dynamic UK trigger

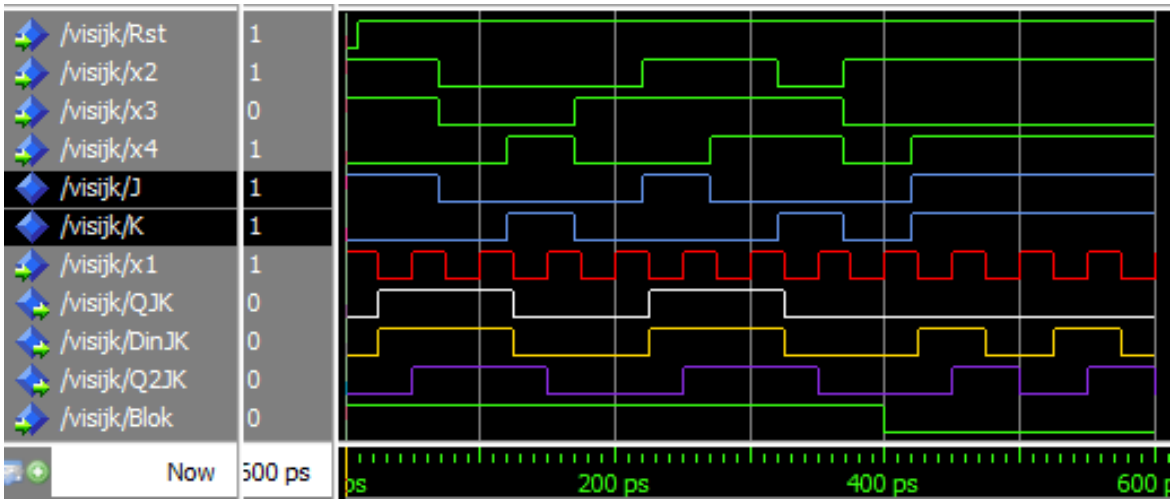


Figure 6: Combined timing diagram for three types of UK triggers

We can see that the timing diagram of a dynamic trigger up to 400 ps (static trigger synchronisation lockout) does not differ from the timing diagram of a static trigger. To highlight the differences, we will slightly modify the test by delaying the generation of the second high level of signal J: instead of 220 ps, this signal will be generated at 240 ps (with the synchronisation signal level active). To do this, the directive

force -freeze sim:/visijk/x2 1 0, 0 70, 1 220, 0 320, 1 370

replace with the following:

force -freeze sim:/visijk/x2 1 0, 0 70, 1 240, 0 320, 1 370

The timing diagram of the dynamic trigger will change (Figure 7):

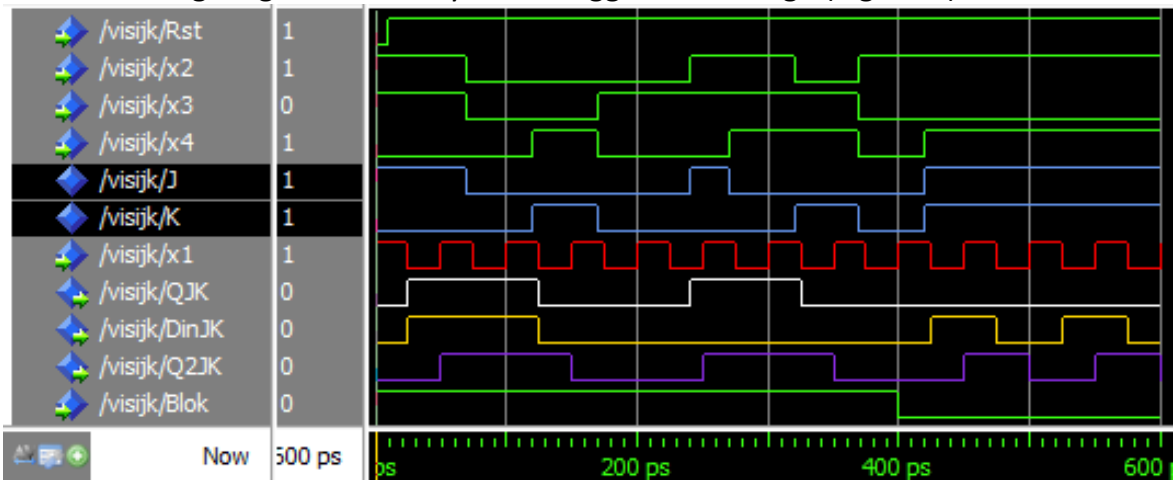


Figure 7. Modified joint timing diagram for three types of UK triggers

Readers are left to compare the timing diagrams of the three triggers, note the differences and draw their own conclusions.

3. Now let's design a 4-bit long universal shift register that performs the following operations:

- recording information;
- information storage;
- a logical leftward push (LL1), accepting D_L in the vacant column;
- logical right shift (LR1), accepting D_R in the column that opens.

We create a table of how the universal register works:

A A_{10}	D_3	D_2	D_1	D_0	Operation
0 0	Q_3	Q_2	Q_1	Q_0	Storage of information
0 1	D_R	Q_3	Q_2	Q_1	Logical right shift by entering D_R (LR1)
1 0	Q_2	Q_1	Q_0	D_L	Logical leftward shift by entering D_L (LL1)
1 1	B_3	B_2	B_1	B_0	Parallel recording of information

The registry runs four micro-operations, so we use multiplexers to manage them. We connect the signals A_1 , A_0 to the multiplexer inputs SD2 and SD1, and the multiplexer inputs D0-D3 to the signals using the register operation table above.

We will use the library synchronous D triggers (*D Flip-Flop GSR Clear*) **fd1s3ax**. For synchronous zeroing, we will insert IR elements controlled by the RESET signal before the D trigger inputs (for asynchronous zeroing, we will choose the **fd1s3dx** triggers). When RESET = 0, the outputs of the IR elements are 0, which are written to the register when the synchronisation signal is applied.

A diagram of the Universal Registry is shown in Figure 8.

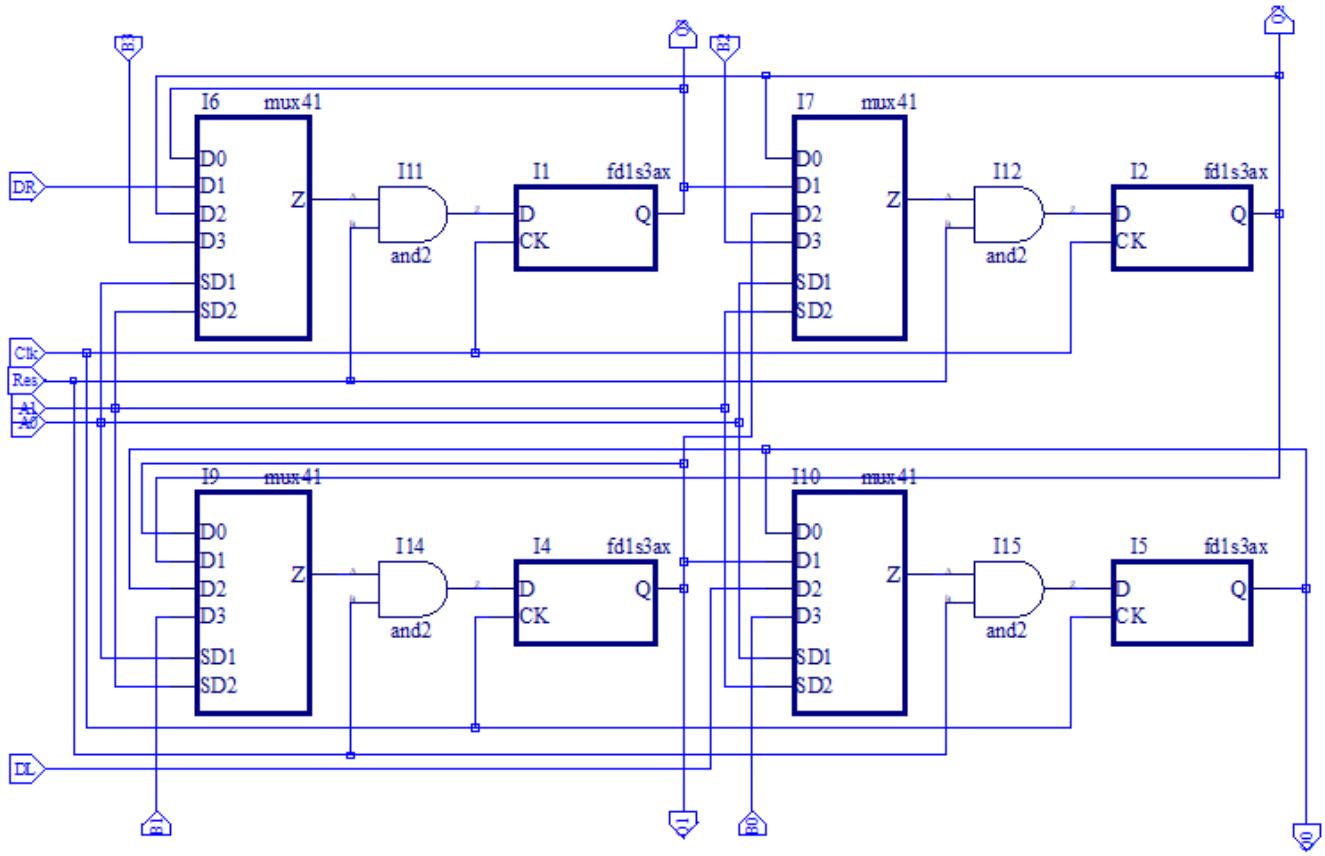


Figure 8: Schematic of the universal shift **register**

We create a test to check the registry's performance, which checks all the micro-operations the registry performs. The simulation produced the results shown in Figures 9a and 9b.

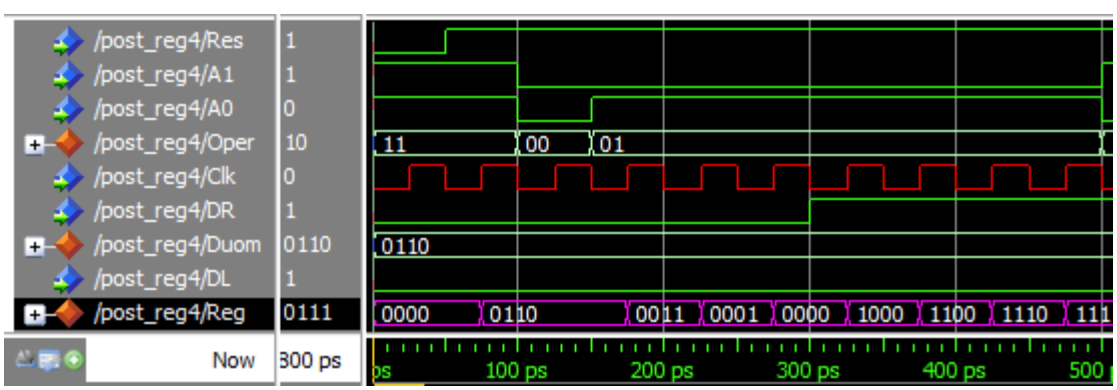


Figure 9a: Recording, storing and right advance

As can be seen, using *Combine Signals,* signals **A1** and **A0** have been combined into **Oper**, input data lines **B3-B0** have been combined into **Duom**, register outputs **Q3-Q0** have been combined into **Reg**. The value of the signal **DR** was shifted to the middle of the

right shift and the value of the signal **DL** was shifted to the middle of the left shift:

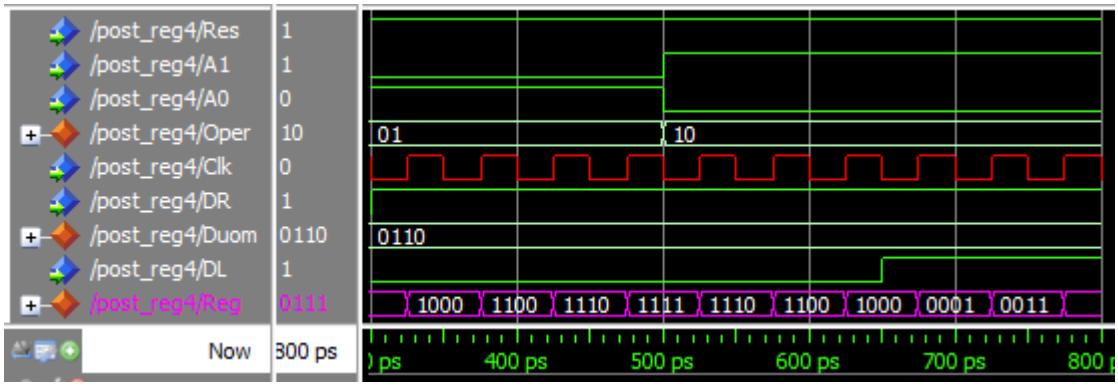


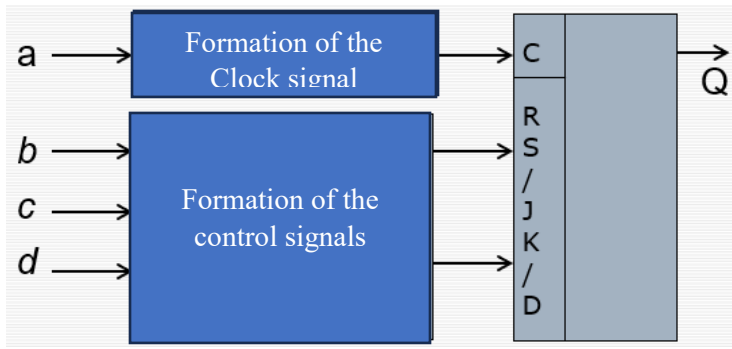
Figure 9b: Information push to the left (from 500 ps)

The change in the **reg** value over time indicates that the shift register is functioning correctly.

<u>Note</u>: If the task specifies asynchronous zeroing, D triggers with asynchronous RESET input (*D Flip-Flop Asynchronous clear*) must be used in the shift register - **fd1s3dx**. These are set to zero by a short high level signal applied to the CD.

Features of trigger testing

A generalized design of the trigger with its control circuitry can be illustrated as follows:



The objective of the test is to verify the correctness of the trigger and its control circuitry by applying all permissible combinations of input signals **a**, **b**, **c** and **d** and monitoring the trigger output **Q**.

The trigger check requires monitoring the values of the output **Q** by feeding the control inputs (D for trigger **D**, **S** and **R** for trigger SR, **J** and **K** for trigger JK) with values that verify that the trigger performs the "1", "0", and the write and store operations correctly.

The synchrosignals shall be separately shaped and the moment of change of the values of the control inputs shall not coincide with the synchrosignal front end.

We will now explain the design of the test using the example of the UK trigger.

Let the following table describe the dependence of the UK signal values on the variables x_2 , x_3 , x_4 :

x_2 x_3 x_4	*K*	*J*
0 0 0	0	0
0 0 1	1	0
0 1 0	0	0
0 1 1	1	0
1 0 0	0	0
1 0 1	1	1
1 1 0	0	1
1 1 1	0	0

When testing a static JK trigger, the combination **J** = **K** = 1 is not possible.

From the table we can see that in our case the JK trigger is set to state **Q** = 1 when the combination of signals $\mathbf{x_2}$, $\mathbf{x_3}$, $\mathbf{x_4}$ is 1 1 0 (then K = 0, J = 1), the trigger is set to state **Q** = 0 when the combination of signals $\mathbf{x_2}$, $\mathbf{x_3}$, $\mathbf{x_4}$ is set to 0 0 1 or 0 1 1 (then K = 1, J = 0) and the trigger is not set to Q0 when $\mathbf{x_2}$, $\mathbf{x_3}$, $\mathbf{x_4}$ is set to 0 0 0, 0 1 0, 1 0 0 or 1 1 1 (then K = 0, J = 0).

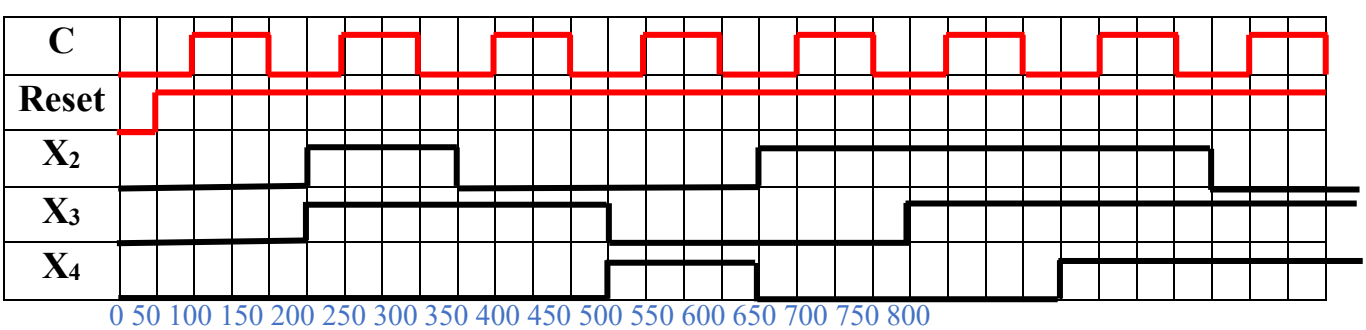
The test is to check that the trigger receives and stores the information (0 and 1) correctly. So the sequence of testing will be as follows (the initial state Q = 0 will be set by the **Reset** signal):

Action	x_2 x_3 x_4
We keep **Q** = 0	0 0 0
We set **Q** = 1	1 1 0
We keep **Q** = 1	0 1 0
We set **Q** = 0	0 0 1
We keep **Q** = 0	1 0 0
We set **Q** = 1	1 1 0
We keep **Q** = 1	1 1 1
We set **Q** = 0	0 1 1

As can be seen, the trigger is set to "1" using the same combination of $\mathbf{x_2}$, $\mathbf{x_3}$, $\mathbf{x_4}$ values 1 1 0, and the trigger is set to "0" using both combinations of $\mathbf{x_2}$, $\mathbf{x_3}$, $\mathbf{x_4}$ values (0 0 1 and 0 1 1). We have used all 4 combinations of $\mathbf{x_2}$, $\mathbf{x_3}$, $\mathbf{x_4}$ values to store the information.

Note that the trigger is initially set to the initial state **Q** = 0 by the **Reset** signal when **C** = **0.** When debugging the scheme, the values of the variables $\mathbf{x_2}$, $\mathbf{x_3}$, $\mathbf{x_4}$ must be changed also when **C** = **0.**

The table should show the values of the signals generated:



The timeline below the diagram assumes a synchronisation period of 100 ps.

We will use the following modelling directives to generate signals:

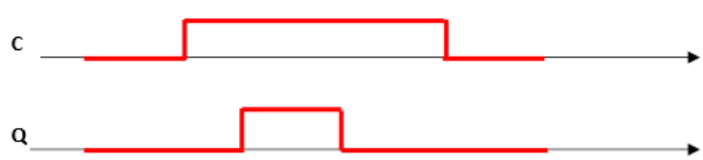
force -freeze sim:/...../C 0 0, 1 {50 ps} -r 100
force -freeze sim:/...../Reset 0 0, 1 {25 ps}
force -freeze sim:/...../X2 0 0, 1 125, 0 225, 1 425, 0 725
force -freeze sim:/...../X3 0 0, 1 125, 0 325, 1 525
force -freeze sim:/...../X4 0 0, 1 325, 0 425, 1 625

How to show the difference between static and dynamic triggers

If the values of signals **x2**, **x3**, **x4** are changed at the passive level of the synchronisation signal, we can see that the timing diagram of the dynamic trigger is not different from that of the static trigger. To highlight the differences in operation, the shaping directives for signals **x2**, **x3**, **x4** need to be changed slightly.

We recommend the following experiments:

1) Let's start with the test as discussed above, and at the end of the test let's add the sets that change the trigger state from "0" to "1" and from "1" to "0" at the active level of the synchronisation signal instead of the passive synchronisation signal level.
2) With the synchronisation signal level active, let's generate D or R,S or J,K signals to form a momentary trigger switch, for example from "0" to "1" and from "1" to "0":



Testing the scheme with all UK triggers

We will now explain the design of the test for a scheme with three types of UK triggers. This is useful to see how the dynamic trigger "gates" when **J** = **K** = 1.

Since such a combination of signals (**J** = **K** = 1) is not possible for the static and the Master-Slave[2] triggers, the following principle can be used: we terminate the

[2] This depends on the feedback configuration of the two-stage UK trigger.

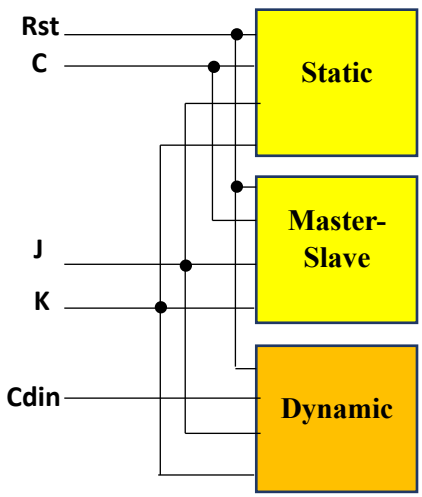
sequence **C** synchronising the static and the Master-Slave triggers before the point in time (say 600 ps) when the signals **J** = **K** = 1 are formed:

force -freeze sim:/visi_jk/C 0 0, 1 {50 ps} -r 100 -cancel 600,

while the dynamic trigger will be synchronised in the sequence **Cdin:**

force -freeze sim:/visi_jk/Cdin 0 0, 1 {50 ps} -r 100.

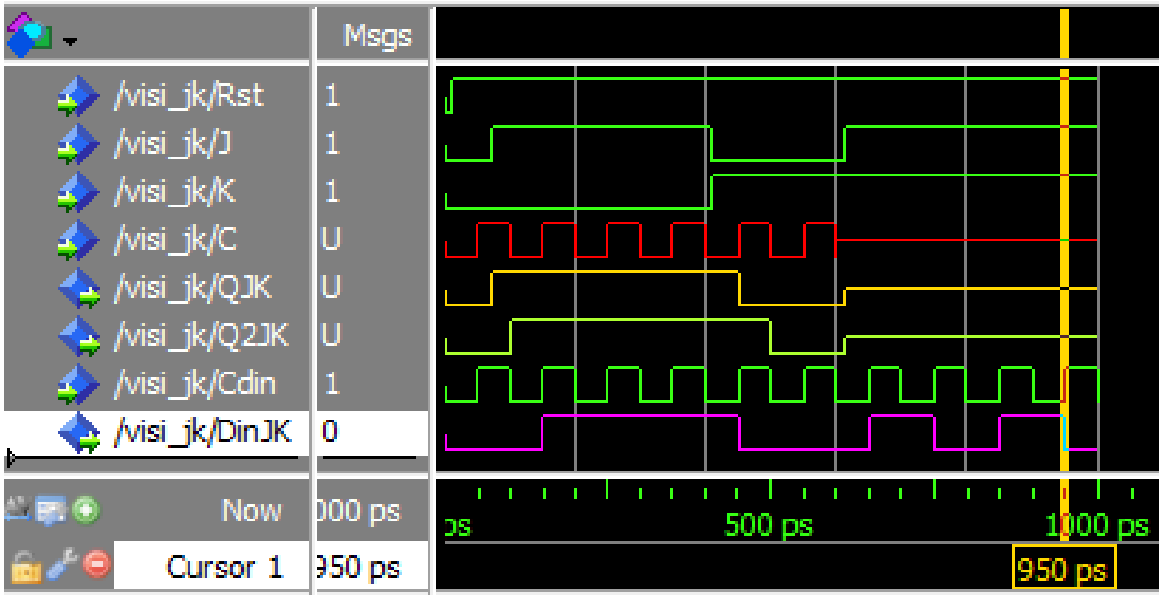
Graphically, this will be represented by the following figure:



To simulate how such a scheme works with directives:

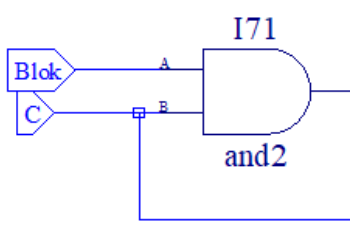
```
restart -f
force -freeze sim:/visi_jk/Rst 0 0, 1 10
force -freeze sim:/visi_jk/C 0 0, 1 {50 ps} -r 100 -cancel 600
force -freeze sim:/visi_jk/Cdin 0 0, 1 {50 ps} -r 100
force -freeze sim:/visi_jk/J 0 0, 1 70, 0 410, 1 600
force -freeze sim:/visi_jk/K 0 0, 1 410
run 1000
```

we got the following picture:



We can see that the static (QJK) and Master-Slave (Q2JK) triggers do not stop the simulation process when the C-sequence is terminated, while the dynamic trigger (DinJK) "gates" when **J** = **K** = 1 and its synchronisation (**Cdin**) is not stopped.

This can be achieved in another way by blocking the static and Master-Slave trigger synchronising sequence **C** before **J** = **K** = 1 is set. To do this, we will add an IR element in the circuit:



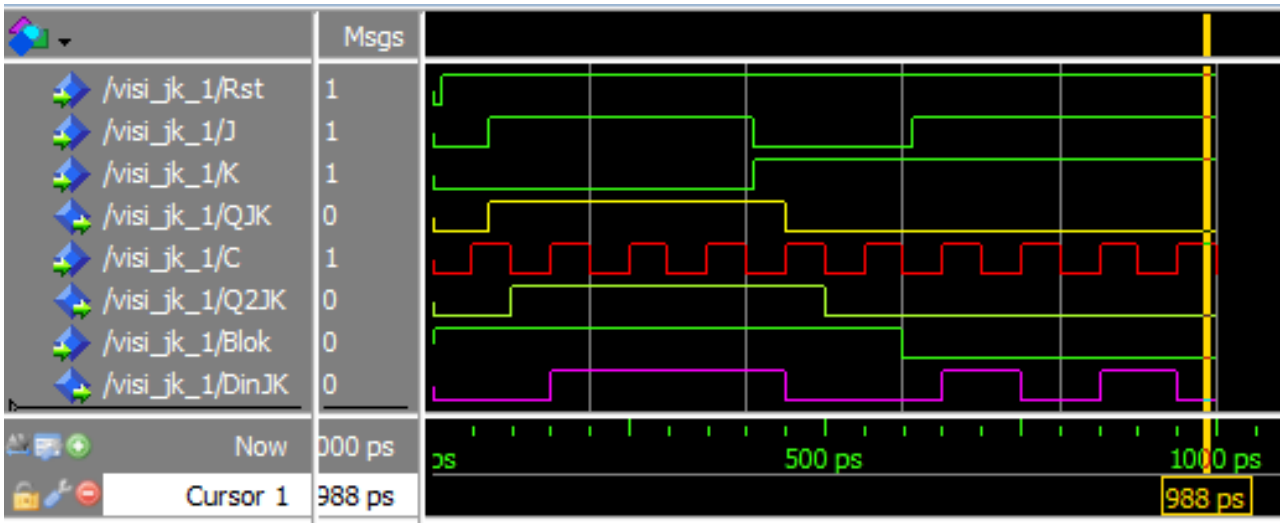
For synchronising static and two-stage triggers

For dynamic trigger synchronisation

To simulate how such a scheme would work with the following directives:

restart -f
force -freeze sim:/visi_jk_1/Rst 0 0, 1 10
force -freeze sim:/visi_jk_1/C 0 0, 1 {50 ps} -r 100
force -freeze sim:/visi_jk_1/Block 1 0, 0 {600 ps}
force -freeze sim:/visi_jk_1/J 0 0, 1 70, 0 410, 1 610
force -freeze sim:/visi_jk_1/K 0 0, 1 410
run 1000

we got the following picture:



So you can choose one or the other.

What to do when the simulation stops

1. This is usually the case when modelling SR and JK trigger operation. Then check what values of signals **R** and **S** (or **J** and **K**) are formed at the moment of stop (i.e. will be at the next moment after the stop); you may have miscalculated the value of a function. The output signals of the above control schemes can be output to the screen (for example, by formalizing them as additional outputs in the scheme and giving them names), but the simulation will be stopped in the event of a malfunction of the trigger.
2. Check that this moment does not coincide with the front of the synchronisation signal.
3. Try "flipping" the synchronisation signal **C.**
4. **The reset** signal must be applied at the passive synchronisation signal level (0 at the trigger input).

If you can't get to grips with modelling, which is usually the case with SR and UK triggers, you can try the following tools:

1. Disconnect the trigger and check only the operation of its control circuits: check what values of signals **R** and **S** (or **J** and **K**) are generated for the combinations of input variables given in the test and compare the results with the values of signals **R** and **S (**or **J** and **K) in** the table you calculated.
2. If you have doubts about the operation of the trigger circuit (maybe you made a mistake when drawing it), copy the circuit on a separate sheet, add

the inputs, and check only its operation by writing a test for its inputs **R** and **S** (or **J** and **K**) and for the synchronization signal **C.**